CSE446/598 Project/Assignment 3 Integration and Messaging (50 Points)

Spring 2026

Assignment 3 Due: Saturday February 28, 2026, by 11:59pm, plus a one-day grace period

---

Introduction

The aim of this project/assignment is to make sure that you have read the text and the slides, and you have understood the software integration and workflow concepts in Workflow Foundation covered in the lectures and in the text. By the end of the project, you should have a good understanding of the concepts and have applied these concepts in developing operational software in Workflow Foundation. The assignment is an individual assignment. No collaboration is allowed. All students must submit independently developed applications. You cannot repeat the examples given in the book or lecture slides. You can follow the examples, but significant changes must be made.

Practice Exercises for Assignment 3 (No submission required)

1. Reading: Textbook chapters 7 and 8 on workflow and on Messaging and on other composition languages. Read Textbook Section 5.5 on Caching.
2. Practice exercises: Questions at the end of Chapters 5 (on Caching), questions at the end of Chapters 7 and 8 (on Messaging and on other composition languages). Complete the multiple choice questions. The solution is available in the course web page.
3. Follow the lectures 2-4 and create a free account of Power Platform.
4. Follow the lectures 2-3 and 2-4, as well the text chapter 7 to implement a Workflow Foundation application and a Power Platform workflow application.
5. Explore the ASU Repository at: https://venus.sod.asu.edu/WSRepository/repository.html
6. Follow the lecture 2-4 to implement a web application using Power Platform and Copilot.
7. Follow the lecture 3-1 to implement a web application using Google Antigravity IDE.
8. Follow the lecture 3-2, the text Chapter 8, and the BPEL tutorials in the assignment folder to implement a BPEL process.
9. Install Oracle SOA Suite and BPEL programming environment.
10. Write a client to consume the BPEL service.
11. Follow Lecture 3-3 to implement a messaging system.
12. Follow Lecture 3-4 to implement a website with caching.

Note: Workflow Foundation module may not be included in your Visual Studio. You need to run Installer to add Workflow Foundation. It is under the tag of Workloads. You can also use your Visual Studio to

open an existing Workflow Foundation. Then, Visual Studio will add Workflow Foundation module automatically.

Assignment Questions: Workflow Composition (Submission Required, 50 points)

This assignment is an individual assignment, and it is an open-ended type of assignment. Each student must perform and submit independent and unique work based on the project requirements.

You will submit two questions listed below. You can choose **one** question from Questions 1, 2, 3 and 4. Question 5 is **required**. When you submit, please indicate in the "Text Entry" part of your choice: 1. Workflow Foundation, (2) Power Platform, (3) Antigravity, **or** (4) BPEL.

1. In this part of assignment, you will develop a sensible application that consists of Presentation (Human user interface) and application (business) logic tier, as shown in Figure 1. For the human user interface, you can use any type: Console, HTML5, MVC, Windows Forms Application, or ASP .Net website application. For the business logic tier, you **must** use Workflow Foundation.

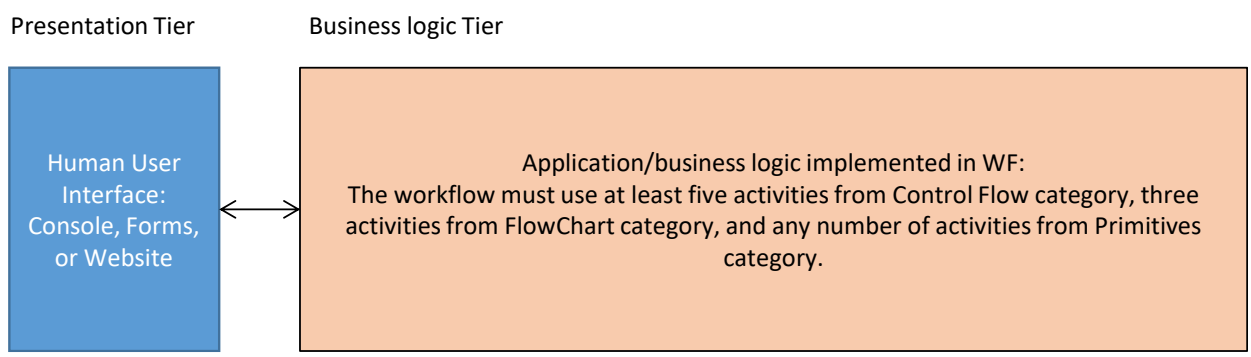


Figure 1. Workflow-based application

In the workflow, you must use at least five different types of activities from Control Flow category, three different activities from FlowChart category, and any number of activities from Primitives category. The categories are shown in Figure 2.

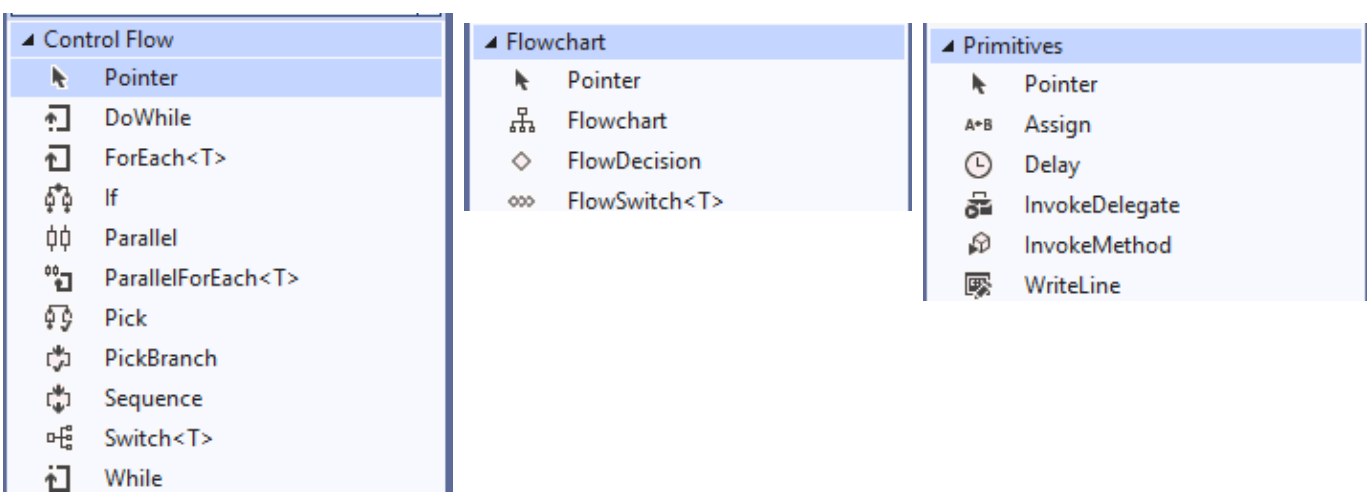


Figure 2. Categories to be used in the workflow

The assignment question is broken down into the following tasks.

1) Human user interface design. [10 points]

2) Workflow design that uses the required activities and forms a sensible business logic. [15 points]

Question 1 Submission: The entire executable solution folder with all the code and project files must be zipped into a file for submission.

2. Use Power Platform to develop a sensible application of your choice. The application must consist of simple UI (User Interface), data source, flow, and visualized output. You can choose any method to start your application: blank, from template, or use Copilot. You can follow the use case given in lecture and online tutorials, but you need to extend the examples significantly to implement you own ideas.

   1) UI design for input [5 points]

   2) Data source: you can use dataverse, Excel, or any type of table. [5 points]

   3) Flow: Your follow must consist of at least five activities. [5 points]

   4) Visualized output: Graphic output is required. [5 points]

   5) Test cases: Execute your application using two test cases of your choice. [5 points]

   Question 2 Submission: (1) Submit screenshots that show all the parts listed above. (2) Optional: Submit the link of your app if it is deployed in cloud.

3. Use Antigravity to develop a sensible web application of your choice. The application must consist of simple UI (User Interface), data source, structured prompts, and visualized output. You can follow the use case given in lecture and online tutorials, but you need to extend the examples significantly to implement you own ideas.

   1) UI design for input [5 points]

   2) Data source: you can use text file, Excel, or any type of disk file or database. [5 points]

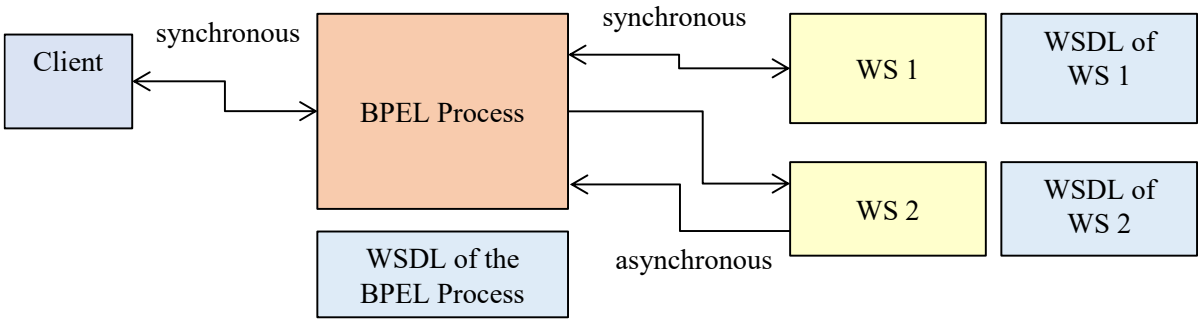
   3) Structured prompts: You must define a sequence of prompts that work together to construct the development of a coherent application. [5 points]

   4) Visualized output: Graphic output is required. [5 points]

   5) Artifact/deliverable: Generate standalone (or deployed application) and deliverable artifacts for submission. [5 points]

   Question 3 Submission: (1) Submit the generated artifacts and the screenshots that show all the parts listed above. (2) Optional: Submit the link of your app if it is deployed in cloud.

4. Use BPEL process to implement a sensible business logic. The required structure is given in Figure 3.



You can simplify it to include the WSDL service call part only. You can directly use an XML editor (for example: https://www.tutorialspoint.com/online_xml_editor.htm, or https://codebeautify.org/xmlviewer) to write the BPEL process and the part of the WSDL required in this assignment. You do **not** need to create the operational executable software using a BPEL development environment, such as Oracle SOA Suite.

1) Give the BPEL process part that calls the WSDL service. It must include: partnerLinks of the BPEL Process, variable needed, the logic (constructs) such as <sequence>, <assign>, <copy>, <invoke>, <reply>, etc. [15 points]

2) Give the WSDL file of the BPEL process. It must include the definition of the partnerLinkType, role, portType. [10 points]

For question 3, submit the parts of the BPEL Process file and BPEL's WESD file required in the question.

Question 1, 2, 3, or 4 Submission: When you submit, please indicate in the "Text Entry" part of your choice: 1. Workflow Foundation, (2) Power Platform, (3) Antigravity, **or** (4) BPEL.

This question is independent of Questions 1, 2, 3 and 4. This question is required in this assignment.

5. Create a simplified messaging service to buffer messages in an XML (or JSON) file and a Web client to allow users to send and receive messages. You can develop your service and application either in C# or in Python, and you must host the messaging service on localhost.

5.1 Messaging service: Develop a RESTful service (in C# or in Python) in Visual Studio that can buffer messages before the receivers can fetch the messages any time. The messages must be saved in an XML file or a JSON file that the messaging service can access. The service must offer the following operations. You may add more parameters for additional functions. [15]

- Operation 1: sendMsg: It allows the client to send a string-type message to the messaging service, and the message will be stored in the database (XML file or JSON file) with at least a senderID, a receiverID, and a time stamp.
  Input parameters: string senderID, string receiverID, string msg, etc.
  Output: void
  Note: a timestamp should be added by the program, and you can decide the data type to use.

- Operation 2: receiveMsg: It allows the client to receive all the messages sent to the receiverID.

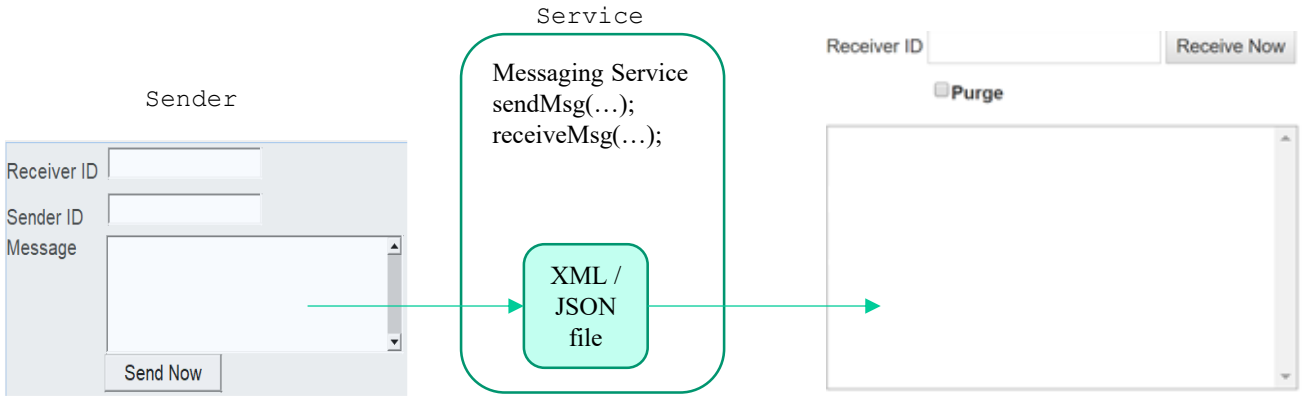
Inputs: string receiverID, boolean purge
Output: string[] an array of messages, with each element containing the related information: senderID, sending time, and message.
The receiver should receive the new messages that have not been received in the previous receiveMsg call. If purge == true, the service will delete all messages for the receiver.
Note:

- New messages are those that have not been retrieved before. This operation must retrieve all the new messages only for each call.
- Old messages are those that have been retrieved before but not deleted (not purged). You should not receive these messages. You do not have to provide a way to re-read the old messages in this assignment.
- Deleted messages cannot be received.

In this assignment question, you are required to receive new messages only. But you could offer some methods to read the old messages -- not required in this assignment. You cannot read the deleted messages.

If you are not familiar with XML (JSON) processing, please reference the sections on reading and writing an XML file on disk, you may want to read text Chapter 4 on XML processing and Section 5.4.2 on reading and writing XML files. You may also read Text Chapter 10 and use LINQ to XML methods.

5.2 MsgApp: Develop a Web application (client) in ASP .Net, MVC or HTML5 that contains at least two Web pages: Sender page and Receiver page. The sender sends messages to the Messaging service and the receiver receives messages from the service. A sample GUI is shown below. The GUI must link to all components that need to be implemented. You can add additional items in your GUI. The application project can be added into the messaging service solution. You can also create a separate solution for this part of the project. [10 points]



To test your clients (sender and receiver) on localhost/IIS Express: Right click the service project and choose View in Browser to run the service. Then, start the client, which will open another bowser window to run the client. Copy and paste the URL into a new browser window. Now, you have three sessions of your application: one service session and two client sessions. You can test your application by sending a message in one browser and receiving the message in another browser. Submit a screenshot of your testing.

Question 5 Submission: Create a folder that includes the solution for both question 5.1 and question 5.2. Zip folder for submission. Make sure that your solution and project can run on TA's computer.

General Submission Requirement

All submissions must be zipped into a single file electronically submitted to the Canvas assignment folder. All files must be zipped into a single file for submission.

For programming assignments, the entire solution folder with all the files must be included. To make sure that you have all the files in the zip file, unzip the file on a different machine or in a different directory, and test if you can run the project in a different location.

Canvas submission notice: The assignment consists of multiple distributed projects and components. They may be stored in different locations on your computer when you create them. When you create the Project/Solution, make sure you choose to save the files into a location for submission. Do not use the default location. If your projects are saved into different locations, you must copy these projects into the submission folder for Canvas submission. To make sure that you have all the files included in the zip file and they work together. You must test them on a different computer before submission. You must also download your own submission from Canvas. Unzip the file on a different location or computer, and test your assignment and see if you can run the solution in a different location, because the TA will test your application on a different machine.

Grading and Rubrics

Each sub-question (programming tasks) has been assigned certain points. We will grade your programs following these steps:

(1) Compile the code. If it does not compile, 50% of the points given for the code under compilation will be deducted. Then, we will read the code and give points between 50% and 0, as shown in right part of the rubric table.

(2) If the code passes the compilation, we will execute and test the code using test cases. We will assign points based on the left part of the rubric table.

In both cases (passing compilation and failed compilation), we will read your program and give points based on the points allocated to each sub-question, the readability of your code (organization of the code and comments), logic, inclusion of the required functions, and correctness of the implementations of each function.

Please notice that we will not debug your program to figure out how big or how small the error is. You may lose 50% of your points for a small error such missing a comma or a space! We will apply the following rubrics to **each sub-question** listed in the assignment. The points assigned to a sub-question is shown in parameter *pts in the table*:

Rubric Table

Major	Code passed compilation				Code failed compilation		
Points	pts * 100%	pts * 90%	pts * 80%	pts * 70% - 60%	pts * 50% or 40%	pts * 30% or 10%	0 or 1 point
For each	Meeting all	Working	Working	Working	Failed	Failed to	**0** points if no submission

sub-question	require-ments, well commented, and working correctly in all test cases	correctly in all test cases. Comments not provided to explain what each part of code does.	with minor problems, such as not writing comments, code not working in certain uncommon boundary conditions.	in most test cases, but with a major problem, such as the code fails a common test case	compilation or not working correctly, but showing serious effort in addressing the problem.	compile, showing some effort, but the code does not implement the required work.	**1** point if submitted files that do not answer any questions required, for example, the files are empty, cannot be open, wrong files, etc.

Late submission deduction policy:

- Grace period (Sunday): Assignment submission due date: Saturday. No penalty for Sunday submission;
- 10% grade deduction for every day after the grace period (Monday and Tuesday submissions)!
- No submission will be allowed after Tuesday 11:59pm. The submission link will be disabled.